



TIBANE LABS

APPLIED CRYPTOGRAPHY & SECURITY RESEARCH

SUPPLY-CHAIN INCIDENT ANALYSIS

An In-House Signer in Production

How SecondFi (formerly Yoroi) came to sign real Cardano transactions with EMURGO's own in-house multi-chain SDK — internally "Dullahan," published to npm as [@stashers.io/trantor](https://www.npmjs.com/package/@stashers.io/trantor) — with no publicly known audit, and the predictable-nonce defect that drained user wallets.

SUBJECT

SecondFi / Yoroi (EMURGO)

COMPONENT

Dullahan (npm: [@stashers.io/trantor](https://www.npmjs.com/package/@stashers.io/trantor))

CLASS

CWE-330/323 · process & provenance

DATE

28 June 2026

AUTHOR

Tibane Labs

SUMMARY

01 What happened

Between 21–23 June 2026, reportedly thousands of Cardano wallets created with SecondFi (the rebranded Yoroi wallet) were drained.¹ The root cause was a predictable Ed25519 signing nonce that leaks a private key from a single transaction.² This report is not about the mathematics of that defect — that is covered in our companion advisory — but about **how the vulnerable code came to run in a production wallet at all**.

The vulnerable signer was **EMURGO's own in-house multi-chain wallet SDK** — known internally as "Dullahan"¹⁵ and published to npm as `astashers.io/trantor`^{5,6,7} by its original author, a **former EMURGO employee** (Boris Kolar / stashers.io), after he left.^{8,9,15,16} EMURGO's own copyright-takedown filing identifies the code as its proprietary "Dullahan" project¹⁵ — a name that appears verbatim in the published source's `DullahanExplorer` module³; the author's counter-filing disputes only its licensing, not its origin.¹⁶ Only one version of the npm mirror (`0.5.2`, pre-1.0) was ever published; it has **no publicly known security audit**, and a Project Catalyst proposal to fund one was never funded.⁷ In SecondFi's June rewrite, this SDK **replaced EMURGO's previously-shipped Rust signing implementation**¹⁰ — the established `cardano-serialization-lib`, which had been signing correctly — and went live to users. The defect first appears on-chain on **2026-06-08** — the day the SDK-based build (`10.0.3`) shipped.^{11,12}

◆ THE FINDING IN ONE LINE

The cryptographic mistake — in code its author, by his own (attributed) account, considered experimental — is forgivable. The incident was a governance failure: a co-founding entity of Cardano took its **own** in-house signer — demonstrably insecure, with no publicly known audit — and promoted it into a custodial-scale wallet, replacing an established library that had been signing correctly, without a one-line self-test that would have caught the defect instantly.

THE DEFECT

02 One missing secret, recapped

Cardano keys are Ed25519-BIP32 extended keys: a 64-byte secret $k_L \parallel k_R$, where k_L is the signing scalar and k_R is the secret nonce-half. A correct signer derives the per-signature nonce as $\text{SHA-512}(k_R \parallel M)$. `trantor`'s hand-rolled signer (`blockchain/crypto/ed25519/curve.ts`)^{3,4} computes it from the message alone:

```
// astashers.io/trantor 0.5.2 - ed25519/curve.ts (TS types elided; sign() identical
to the shipped 10.0.3 build)
export function sign(message, keys) {
  const h = sha512(message)                                // ← nonce = SHA-512(message); kR is
  never used
  const p = publicKey(h)                                    // R = (h mod L) · B
  const s = _scalar(sha512(bytes(p, publicKey(keys.secret), message)))
```

```
return bytes(p, _bytes((_scalar(h) + s * _scalar(keys.secret)) % ed25519.CURVE.n))
}
// correct would be: const h = sha512(bytes(kR, message))
```

Because M (the transaction body hash) is public, the nonce is public, and the private scalar falls out of one signature: $\alpha = (s - \text{SHA-512}(M)) \cdot H^{-1} \bmod L$. Every address that signed even once leaked its key. (Reinforcing the bug, the key model also slices the 96-byte extended secret down to kL only — `Bytes(96, 0, 32)` — so `kR` never even reaches the signer.)⁴

THE COMPONENT

03 What trantor is

`astashers.io/trantor` is the npm-published form of "**Dullahan**" — an in-house "Multi-Chain Wallet SDK for Cardano, Bitcoin & Ethereum"⁵ developed at EMURGO and published to npm, after he left, by its original author **Boris Kolar** (`stashers.io`).^{15,16} The npm mirror was the subject of a **Project Catalyst (Fund 15)** proposal requesting funding to finish and audit the SDK — including a **10% line (10,000 ADA) for a security audit**. That proposal was **never funded**: Fund 15's community vote never took place — the entire fund was paused amid a Project Catalyst stewardship transition — so the proposal remains in "pending vote" status. The requested audit was contingent on a vote that never happened. In short: **EMURGO's own in-house SDK, with no publicly known audit, whose only published version (0.5.2) is pre-1.0.**^{6,7} Its public footprint:^{8,9}

SIGNAL	STATE
npm package <code>astashers.io/trantor</code>	single version 0.5.2, published 2025-08-25
Declared repository / homepage in package	none
Source repo <code>github.com/stashers-io/trantor</code>	404 — EMURGO DMCA target (Aug 2025); now offline/private
npm tarball <code>astashers.io/trantor@0.5.2</code>	still published — not taken down (this report's source)
Org repo <code>github.com/stashers-io/stashers</code>	1 commit — a <code>.gitignore</code> only
Author profile <code>github.com/boris-kolar</code>	0 public repositories
Catalyst Fund 15 proposal (the audit funding)	pending vote — fund paused; never voted, never funded
Completed independent audit	none

The package itself declared no repository link, and the source repository that did exist (`stashers-io/trantor`) is **no longer accessible** — it was EMURGO's DMCA target and now returns 404.^{15,16} The copy that remains publicly retrievable is the **npm tarball** (`astashers.io/trantor@0.5.2`), which was **not** taken down and is this report's reproducible source. No independent audit is publicly known; the only contemporaneous maturity signal was the pre-1.0 version number.

Provenance — EMURGO's own code, disputed only on licensing

That the code is EMURGO's own is established by EMURGO itself. In GitHub's public DMCA archive (record dated **26 Aug 2025**), **EMURGO Group Pte. Ltd.** filed a takedown against the **GitHub repository** `stashers-io/trantor` (now offline — HTTP 404; the npm tarball was left untouched), calling the entire repository an unauthorized copy of its **proprietary "Dullahan"** SDK by "one of our former employees."¹⁵ The author filed a **counter-notice** (record dated **9 Sep 2025**) disputing not the origin but the licensing — asserting the project carried an `MIT LICENSE.txt` added **6 June 2024** and that some contributed code predated his EMURGO contract.¹⁶ Two points hold independently of either party's legal position: the name **"Dullahan"** in EMURGO's own filing matches the `DullahanExplorer` module in the published source³ (so the npm package is the Dullahan codebase), and the deployed app is `com.emurgo` — EMURGO's own. This was therefore **not a third-party dependency that slipped in**: it was EMURGO's own in-house code reaching EMURGO's own production app. Whether the production build pulled the public npm package or built from EMURGO's internal Dullahan repository is **undetermined** — both are the same codebase, and the deployed `sign()` matches it byte-for-byte.³ The ownership/licensing dispute itself remains legally unresolved.

THE DEPLOYMENT

04 How it reached production

SecondFi's mobile app is React Native. Until spring 2026 it signed through EMURGO's Rust stack (`cardano-serialization-lib` via the `react-native-haskell-shelley / csl-mobile-bridge` native libraries) — the established Cardano standard, and demonstrably correct: the same wallets' pre-June-8 signatures all carry proper nonces. The June rewrite removed that native stack and replaced it with the `trantor` JavaScript signer. The timeline is unambiguous:^{10,11,12}

2025-08-25	<code>trantor 0.5.2</code> published to npm — its only published version (pre-1.0); no public audit.
2026-03-16	Last public Yoroi release tag (v7.0.5); signer is the established Rust CSL — correct nonce .
2026-03-20	App store build 7.0.4 — still ships <code>libreact_native_haskell_shelley.so</code> (Rust).
2026-04 → 05	Yoroi rebrands to SecondFi; the source repository goes dark (last public commit 2026-05-13).
2026-06-08	SecondFi 10.0.3 ships . Rust CSL libraries removed; <code>trantor</code> JS signer is now live. First predictable-nonce signature appears on-chain the same day .
2026-06-21 → 23	Mass drain across affected wallets.
2026-06-24	SecondFi confirms a "deterministic nonce derivation flaw"; app placed in maintenance.

The version number jumped from 7.0.4 straight to 10.0.x, and the build that introduced the regression — 10.0.3 — was **never publicly tagged or open-sourced**.¹ The prior implementation was signing correctly; it was removed and replaced with EMURGO's own in-house SDK (Dullahan, a.k.a. `trantor`).

ATTRIBUTION

05 The deployed signer is `trantor`'s code

We obtained the shipped Android build (10.0.3), extracted the Hermes JavaScript bundle, and decompiled it. Two independent axes establish that the deployed signer is `trantor`'s code.³ One caveat first: we cannot pin the exact version — the shipped build may be a later revision than the npm-published 0.5.2, so 0.5.2 is best read as the basis of what ran. What we can show is that the specific vulnerable `sign()` is unchanged between 0.5.2 and the shipped build.

1 • The vulnerable function decompiles line-for-line

The shipped `sign()` maps instruction-for-instruction onto `trantor`'s published 0.5.2 `curve.ts` — same nonce `sha512(message)`, same `publicKey(h)`, same challenge `sha512(R ⊗ A ⊗ message)`, same $(h + s \cdot \alpha) \bmod n$. The vulnerable function is therefore identical in the published package and the shipped app.

2 · The whole SDK is embedded, not just one file

Of **248 distinctive identifiers** sampled from trantor's published 0.5.2 source (across all ten modules), **221 (89%)** are present in the shipped bundle — including trantor's unique plugin architecture, its Yoroi backend bindings, its hand-rolled curve math, and its self-contained CBOR / bech32 / schema codecs. This is not reuse of a snippet; the app is built on trantor itself.

TRANTOR MODULE	FOUND / TOTAL	REPRESENTATIVE MATCHES
cardano	118 / 129	YoroiBroadcaster, CardanoTransactionSigner, *Plugin
blockchain	58 / 64	addMul8, pointAdd, _pointFromY, _neuter
cbor	19 / 19	Indefinite, Tagged, Stream
cryptography	8 / 8	Hardener, Argon2, Cipher
schema / core / bech32	18 / 18	_polymod, _checksum, _precompute
network / store / wallet	20 / 30	Torrent, SQLite, AddressBook

Per-module rows count an identifier once for **each** module it appears in, so they total more than the 248 distinct identifiers in the headline. ~11% of sampled identifiers are absent because they are TypeScript type/interface names (no runtime presence) or were tree-shaken — the expected residue. The runtime match across ten independent modules removes any doubt that the deployed code is trantor (the exact revision aside).

THE AUTHOR'S ACCOUNT

06 "A hack I wasn't confident in"

After the incident, an X account — `@boris_kolar` — presenting as the SDK's author posted a candid account of the defect.¹⁴ **We have not independently verified that this account belongs to the SDK's author** (the package's npm and GitHub identities declare no associated X handle, and the name match alone is not proof). We therefore reproduce the statements as attributed, not confirmed. With that caveat, they are notable for their candor and align with what the code and package metadata independently show:

"@noble/curves is a good library, no fault there. But it doesn't work as is for Cardano signing. I wrote a few hacks to make it work. It was a hack I wasn't at all confident in and planned to replace. I wanted it to be open source and audited."

"The code was experimental, unfinished & unmaintained. I wanted audit."

"I do feel embarrassed for my error. Had I continued to maintain the SDK, I would have caught it (I was aiming for byte-to-byte compatibility with the old Yoroi). Sincere apologies. I own my mistakes."

— Posts by X account `@boris_kolar`, presenting as the SDK's author · June 2026 · **identity not independently verified** · post URLs & archive.is copies in §10 (ref 14)

Two points in those posts are **independently verifiable** — true regardless of who wrote them. First, **@noble/curves is not at fault**: trantor only borrows noble's curve arithmetic; the broken nonce is in trantor's own hand-written wrapper, which noble's safe API would never have produced. Second, the code **had no publicly known audit and only a single pre-1.0 release** — its npm/GitHub footprint and the never-funded Catalyst audit proposal show this independently of any statement. None of this is the posture of a vetted production dependency.

ACCOUNTABILITY

07 Where the process failed

It is important to separate the error from the incident.

The error (developer)

- A wrong nonce in the signer — a genuine mistake. An audit was pending (the author's Catalyst proposal sought one).⁷ The "hack / unfinished" descriptions are the author's own, post-incident words¹⁴; the published package itself carried no such warning (empty README, no description).⁶
- Publicly addressed afterward (via an account presenting as the author); noble correctly exonerated.
- This is ordinary pre-release risk: unaudited, pre-1.0 code is allowed to be wrong.

The incident (deployer)

- A co-founding Cardano entity¹³ **shipped its own in-house SDK to production**, replacing an established signer that was working correctly.
- It ran **without any independent audit** — the SDK's only path to one (an unfunded Catalyst proposal) was never voted on or funded.
- No **self-test** on produced signatures (a single $\text{SHA-512(M)} \cdot B \neq R$ assertion catches it instantly).
- No **differential test** against the old Rust signer it replaced.

✗ THE SYSTEMIC FAILURE

Custodial-scale crypto deployment is a supply-chain discipline. The defect was trivially detectable — one test vector, or one signature self-check, or a diff against the outgoing implementation would each have stopped it. The failure was not that a dependency contained a bug; it was that an unvetted in-house signer was promoted to production — over an established one that was working — with none of those gates in place.

IMPACT & GUIDANCE

08 Exposure window

From on-chain measurement, fingerprinted (predictable-nonce) signatures exist only between **2026-06-08** and the patch — none before.¹² The user-facing rule is therefore simple:

⚠ IF YOU USED SECONDFI / YOROI

If your wallet **signed any transaction on or after ≈ 8 June 2026**, treat its keys as exposed and move funds to a freshly generated wallet from known-good software. Transactions before that date used the correct (prior) signer and are not exposed by this flaw. Re-importing the recovery phrase into another wallet does **not** help — the exposure is at the key level.

RECOMMENDATIONS

09 Preventing the next one

For wallet operators

- Never swap a proven, established signer for an unvetted one; gate crypto-dependency changes on a completed, independent audit.
- Ship a **produced-signature self-test**: assert $\text{SHA-512(M)} \cdot B \neq R$ on every signature.
- Run **differential tests** against the prior implementation and known test vectors before swapping.
- Prefer vetted primitives end-to-end; never hand-roll EdDSA nonce derivation.

For the ecosystem

- Treat any dependency without a completed, publicly known audit — or shipping pre-1.0, or without published source — as a hard deployment blocker, not an advisory note.
- Fund audits before, not after, code reaches users who hold real value.
- Require reproducible, source-available builds for custodial-scale wallets.

SUBSTANTIATION

10 Evidence & sources

Every load-bearing claim in this report is reproducible from primary sources: the **Cardano mainnet**, the **published npm package** (`npm pack @stashers.io/trantor@0.5.2`), the **shipped Android build** (SecondFi 10.0.3, XAPK $\rightarrow$ `com.emurgo.apk` $\rightarrow$ `assets/index.android.bundle`, decompiled with `hermes-dec`), and **public records** (npm registry metadata and GitHub's public DMCA archive). Numbered markers in the text refer to the items below.

- [1] **Incident & "deterministic nonce" root cause.** SecondFi official statement (x.com/secondfiapp/status/2070034865832202281, Jun 2026; archived archive.is/LNU5v) describing a "deterministic nonce derivation flaw" and the 21–23 Jun 2026 drains; corroborated by independent on-chain reconstructions (Bitquery) and press (CoinDesk).
- [2] **Predictable nonce $\rightarrow$ single-signature key disclosure (verified on mainnet).** For affected signatures, $\text{SHA-512(M)} \cdot B == R$ (the nonce is public), and $\alpha = (s - \text{SHA-512(M)}) \cdot H^{-1} \bmod L$ recovers a scalar with $\alpha \cdot B == A$. Verified e.g. on tx 4655145484...ce02203. Tooling/self-tests: `tools/recover.py`, `nonce_scan.py` (no false positive on correct RFC-8032 signatures).
- [3] **The deployed signer is trantor's code (two independent proofs).** (a) The shipped bundle's `sign()` decompiles line-for-line to trantor's `ed25519/curve.ts` (identical to published 0.5.2). (b) 221/248 (89%) of identifiers sampled from published 0.5.2 across all ten modules are present in the bundle (e.g. `addMul18`, `pointAdd`, `_pointFromY`, `YoroiBroadcaster`, `*Plugin`). Reproduce: `grep -a the bundle bytes`. Caveat: this shows the deployed code is trantor and that its `sign()` equals 0.5.2's; it does not establish the exact shipped version (which may post-date 0.5.2).
- [4] **The defect, in source.** `@stashers.io/trantor@0.5.2, blockchain/crypto/ed25519/curve.ts  $\rightarrow$  sign(): const h = sha512(message); and crypto.ts  $\rightarrow$  key: Bytes(96,0,32) (keeps kL, drops kR).` Reproduce: `npm pack`.
- [5] **"Multi-chain wallet SDK."** Catalyst Fund 15 proposal title "Multi-Chain Wallet SDK for Cardano, Bitcoin & Ethereum"; package ships both Cardano (ed25519) and secp256k1 (BTC/ETH) crypto modules (`blockchain/crypto/{ed25519,secp256k1}`).
- [6] **Early-stage / publication indicators (facts, not a maturity verdict).** Only one npm version was ever published — 0.5.2 (pre-1.0), 2025-08-25 — and the org's only public repo github.com/stashers-io/stashers is a single 2025-08-22 commit containing only a `.gitignore` (source never published). The published package itself carried **no** "experimental" / "do not use" warning — empty README, no description or keywords; the only contemporaneous status signal was the pre-1.0 version number. The "experimental / unfinished / unmaintained" wording is the author's own post-incident, attributed characterization, not a Tibane finding, and the maturity of the exact build the app shipped is not independently established (we hold a 10-month-old published version, 0.5.2).
- [7] **No publicly known audit + Catalyst status.** We found no published audit and the package ships none; the Catalyst Fund 15 proposal that budgeted a security-audit line was **never funded** (Fund 15's vote, orig. 13–27 Jan 2026, never occurred — the fund was paused during a Project Catalyst stewardship transition — leaving it at "pending vote"). Absence of a public audit does not by itself prove no private audit occurred. Source: projectcatalyst.io/funds/15.
- [8] **npm metadata.** `registry.npmjs.org/@stashers.io/trantor: versions {0.5.2}; repository/homepage/bugs = null; maintainer stashers.io <boris.kolar.mail@gmail.com>; published 2025-08-25.`
- [9] **GitHub footprint.** `gh api: repos/stashers/trantor  $\rightarrow$  404; orgs/stashers  $\rightarrow$  404; users/boris-kolar  $\rightarrow$  0 public repos, twitter_username = null; clone of stashers-io/stashers  $\rightarrow$  one .gitignore commit, author Boris Kolar <boris.kolar.mail@gmail.com>.`
- [10] **Rust $\rightarrow$ JS replacement.** APK 7.0.4 (Mar 2026) ships `libreact_native_haskell_shelley.so + libcls_mobile_bridge_jsi.so` (Rust CSL) and its JS bundle contains `signAndAddVkeySignature / toRawKey / Bip32PrivateKey`; APK 10.0.3 (Jun 2026) contains **none** of these — the Rust signing libraries are gone and signing is trantor JS.

- [11] **Build & release timeline.** git tags (v7.0.5 = 2026-03-16, last public); last public commit 2026-05-13; app-store builds (APKPure/apkcombo version history): 7.0.4 2026-03-20, 10.0.3 2026-06-08, 10.0.5 06-11, 10.0.6 06-22.
- [12] **Exposure window (on-chain measurement).** Across 25 sampled victim wallets, fingerprinted signatures span **2026-06-08 → 2026-06-25**, none earlier — despite years of prior (correctly-signed) history. First fingerprinted tx 0d7630eb...258d1 @ 2026-06-08 13:18 UTC. Tooling: tools/find_window.py, victim_timeline.py.
- [13] **EMURGO.** One of the three founding entities of Cardano (with IOG and the Cardano Foundation); publisher of Yoroi → SecondFi.
- [14] **X account identity — unverified; post URLs & archives.** The §6 quotes are three posts by x.com/boris_kolar — status 2070549316989595950, 2070549805428797667, 2070550870824042626 — archived on archive.is at archive.is/pFckt, archive.is/sNbmy, archive.is/VyuNZ respectively. No authoritative cross-link confirms this account is the SDK author (GitHub twitter_username = null; stashers.io links to no X account; a matching display name is not proof) — quoted as attributed only.
- [15] **EMURGO DMCA takedown — code identified as proprietary "Dullahan".** GitHub's public DMCA archive, record dated 2025-08-26: github.com/github/dmca/blob/master/2025/08/2025-08-26-emurgo.md. EMURGO Group Pte. Ltd. ("the rightful owner of proprietary software project Dullahan") asserts the repository github.com/stashers-io/trantor contains "unauthorized copies of our proprietary code" by "one of our former employees" and that "the entire repository is infringing." This is EMURGO's own claim that trantor = its Dullahan project; the name matches the DullahanExplorer module in the published 0.5.2 source (ref 3).
- [16] **Author counter-notice — disputes licensing, not origin.** GitHub's public DMCA archive, record dated 2025-09-09: github.com/github/dmca/blob/master/2025/09/2025-09-09-emurgo-counternotice.md. The repository owner disputes the takedown ("I am the content owner"), asserting the work is MIT-licensed (LICENSE.txt "present since June 6th 2024"), that the README describes Dullahan as "open source," and that "some of the code ... was created before [the author] was contracted by EMURGO." Names/contact are redacted "[private]" in the public record; npm metadata (ref 8) attributes the package to stashers.io <boris.kolar@mail@gmail.com>. A DMCA takedown reflects the filer's assertion (GitHub does not adjudicate ownership); both filings agree the code originated as EMURGO's "Dullahan" project, while ownership/licensing remains legally unresolved.

Prepared by **Tibane Labs** — applied cryptography & security research · tibane.net. Cryptographic and binary claims were verified against Cardano mainnet, the published npm package `astashers.io/trantor@0.5.2`, and the shipped SecondFi Android build 10.0.3. The statements in §6 are quoted from posts by the X account `@boris_kolar`, which presents as the SDK's author; we have **not independently verified that identity** and reproduce them in good faith, clearly attributed. Where this report relies on a claim, the claim is independently verifiable from the chain, the npm package, or the binary; statements of intent or attribution that are not are flagged as such. This document analyses process and provenance; the cryptographic mechanism is detailed in our companion advisory.